SMART CAMPUS NAVIGATION SYSTEM AND SHORTEST PATH

ANGGOTA KELOMPOK

• Maizan Arifin	25051030009
• Habibi Aulia Rizki	25051030010
• Nicholas Candra Pascalis	25051030011
• Rahmatullah Wahyu Nugroho	25051030038
• Juan Ramadrani	25051030039

LATAR BELAKANG

Universitas Negeri Yogyakarta memiliki lebih dari 30 gedung dan fasilitas yang tersebar di area kampus seluas ±100 hektar. Dengan area yang begitu luas dan kompleks, mahasiswa baru sering kali mengalami kesulitan dalam menemukan rute optimal menuju gedung kuliah, terutama ketika dihadapkan pada beberapa alternatif koridor atau jalan yang bisa dilewati. Untuk mengatasi permasalahan tersebut, sebuah sistem navigasi digital berbasis Graph sangat diperlukan karena dapat membantu mahasiswa menemukan jalur terpendek sekaligus memberikan informasi terkait gedung secara cepat. Oleh karena itu, proyek "Smart Campus Navigation & Shortest Path System" ini dikembangkan untuk memodelkan tata letak kampus, di mana setiap gedung direpresentasikan sebagai node (titik) dan koridor atau jalan penghubung direpresentasikan sebagai edge (sisi) berbobot berdasarkan jarak dalam satuan meter. Dalam sistem ini, algoritma Dijkstra digunakan untuk mencari rute yang paling optimal, penjelajahan BFS/DFS digunakan untuk mengeksplorasi konektivitas jaringan jalan, dan struktur data Binary Search Tree (BST) diterapkan untuk menyediakan direktori pencarian gedung yang efisien.

TUJUAN

1. Menciptakan sistem navigasi digital interaktif yang mampu memodelkan tata letak gedung dan jalan menggunakan struktur data Graph.

2. Merancang dan mengimplementasikan struktur data fundamental (seperti Linked List, Stack, Queue, BST, dan Graph) dari nol tanpa menggunakan library struktur data bawaan bahasa pemrograman.

3. Menerapkan algoritma Dijkstra pada Graph berbobot untuk mengkalkulasi dan menemukan rute dengan jarak terpendek antar fasilitas kampus.

4. Mengaplikasikan algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS) untuk mengeksplorasi jalur dan mendeteksi apakah terdapat gedung atau area yang terisolasi dari jaringan utama kampus.

5. Menggunakan struktur Binary Search Tree (BST) untuk membuat direktori pencarian nama gedung agar beroperasi dengan efisiensi waktu logaritmik.

KOMPONEN UTAMA

1.STRUKTUR DATA (UNTUK MENYIMPAN DATA)

- Adjacency List berbasis Linked List

  Digunakan untuk memetakan seluruh kampus. Gedung (seperti Rektorat, FIB) bertindak sebagai Node, sedangkan jalan raya/koridor bertindak sebagai Edge.

- Binary Search Tree (BST)

  Berfungsi sebagai direktori kampus.

- Queue & Stack (berbasis Linked List)

  Keduanya diimplementasikan dari nol untuk menjadi "mesin penggerak" bagi algoritma penjelajahan (BFS dan DFS).

2.ALGORITMA (UNTUK MEMPROSES DATA)

- Algoritma Dijkstra

  Digunakan pada fitur JALUR. Ketika pengguna ingin pergi dari "FIB" ke "Gerbang Utama", Dijkstra akan mengevaluasi semua kemungkinan rute, menjumlahkan jarak (bobot) jalanan, dan selalu memilih jalur yang total jaraknya paling kecil. Hasilnya direkonstruksi menggunakan array parent untuk mencetak urutan gedung yang harus dilewati.

- Breadth-First Search (BFS)

  Digunakan untuk menjelajah dan juga sebagai logika utama fitur TERISOLASI. Sistem akan melakukan BFS dari Gerbang Utama. jika ada gedung yang tidak tersentuh oleh BFS tersebut, artinya gedung itu terisolasi

- Depth-First Search (BFS)

  Digunakan pada fitur JELAJAH_DFS untuk melihat jalur alternatif atau memetakan bagaimana seseorang bisa berkeliling kampus menyusuri satu jalur panjang tanpa henti.

DEMO

ANALISIS BIG - O

Algoritma	Fungsi	Kompleksitas Waktu	Kompleksitas Ruang	Keterangan
QueueLL dan StackLL	Enqueue()	O(1)	O(n)	Penambahan diujung linked list untuk mengubah pointer. Ruang linier terhadap elemen (n)
	Dequeue()	O(1)	O(1)	Mengambil dari depan tanpa menggeser elemen lain
Graph(Adjacency List)	add_node()	O(1)	O(V)	Input ID ke dictionary
	add_edge()	O(1)	O(E)	Add elemen di head linkedlist. E merupakan total jalan
	Neighbors()	O(deg(u))	O(deg(u))	Read semua tetangga dari node.deg(u)=jumlah jalan yang terhubung menuju gedung
Mencari Rute	Dijkstra()	O(V^2+E)	O(V)	Menggunakan array linier untuk mencari jarak minim.
	Reconstruct _path()	O(V)	O(V)	Menelusuri balik pointer parents dari asal
Traversal Graph	Bfs()	O(V+E)	O(V)	Setiap Gedung dan jalan dievaluasi
	Dfs()	O(V+E)	O(V)	Mengevaluasi dengan stack
BST	Insert()/search()	O(log V)	O(V)	Karena pohon miring(tidak seimbang)
	Inorder()	O(V)	O(H)	H=call stack rekursif saat mengunjungi semua node).
Set	Terisolasi	O(V)	O(V)	Operasi pengurangan 2 set (semua_gedung-terjangkau)

KEPUTUSAN DESAIN

- Peta Kampus sebagai Adjacency List

kami memodelkan graf menggunakan Linked List manual, alasannya karena peta kampus adalah graf renggang (sparse graph). Menyimpan 30 node dan 55 jalan dalam list menghemat memori secara signifikan dan membuat penambahan rute berjalan konstan di O(1)

- Pencarian Rute dengan Dijkstra Berbasis Array

Kami menggunakan struktur Array/Dictionary sederhana tanpa Min-Heap eksternal.alasannya adalah karena untuk skala 30 fasilitas, kompleksitas O(V^2 + E) menghasilkan eksekusi nyaris instan tanpa overhead pemeliharaan struktur tree tambahan, sekaligus memenuhi batasan implementasi mandiri (no library).

- Direktori Kampus dengan Binary Search Tree (BST)

Kami memisahkan penyimpanan data nama gedung ke dalam struktur BST, alasannya adalah untuk menurunkan kompleksitas pencarian dari linear O(V) menjadi logaritmik rata-rata O(log V)

- Eksplorasi Jaringan dengan Queue & Stack Manual

Kami membangun antrean (BFS) dan tumpukan (DFS) murni menggunakan Linked List, alasannya adalah untuk memastikan operasi push/pop stabil di $O(1)$ untuk mendeteksi area kampus yang terisolasi dengan cepat tanpa memori tambahan (Dynamic Array).

PEMBAHASAN EFISIENSI

Arowwai Industries

1. ***Min-Heap (Priority Queue)***

Penggunaan Min-Heap bisa memangkas kompleksitas waktu dari $O(V^2)$ menjadi $O(E \log V)$. Mengingat masalah mencari node dengan jarak minimum menggunakan nested loop

Dengan penggunaan ini, kecepatan Djikstra dapat meningkat dari mencari manual berulang-ulang berubah menggunakan struktur data pohon biner

2. ***Operasi penyambungan String pada BFS/DFS***

Masalah pada fungsi "*neighbours*" menyebabkan alokasi memori berulang-ulang yang tidak perlu. Maka dari itu kita dapat melakukan iterasi langsung (traversal) melewati pointer "*next*" dari *EdgeNote* tanpa membuat list baru terlebih dahulu
Laporan Graph sekarang menyisir pointer next secara langsung pada objek EdgeNode.

3. ***Duplikasi pengecekan pada DFS***

Di fungsi dfs, pengecekan "if u not in visited" baru dilakukan setelah node dikeluarkan dari Stack. Hal ini membuat banyak node duplikat masuk ke dalam Stack, memboroskan memori. Untuk menghadapi masalah ini dengan cara menandai node sebagai visited tepat sebelum dimasukkan (push) ke dalam Stack, mirip dengan logika pada BFS.

4. ***Type Hinting & Edge Cases***

Penggunaan komentar # type: ignore pada Queue dan Stack menunjukkan adanya masalah typing. Selain itu, fungsi reconstruct_path menggunakan .get() yang bisa mengembalikan None jika node tidak terjangkau, sehingga berpotensi memicu infinite loop atau eror jika tidak ditangani dengan ketat.
Untuk melakukan efisiensi pada masalah ini kita dapat Membersihkan seluruh kode dari trik kotor # type: ignore dengan memperbaiki inisialisasi pointer bernilai kosong (Optional).

THANK YOU